[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Tree based Genetic Programming

University of Essex
School of Computer Science and Electronic Engineering

Learning objective

- Discuss different ways to **represent programs in tree-based GP**
- Describe **closure** and **sufficiency** in the context of primitive sets
- Describe different way to **initialize** programs
- Discuss issues with **program execution** including **macros** and **interpreters**
- Describe **crossover** with **tree-like representation**.
- Describe and discuss **multi-tree representations** and **automatically defined functions**.

Before we start…

Any questions about coursework assignment?

Prefix Notation (1)

- GP trees and the corresponding expressions can be represented in **prefix notation**.
- In this notation, **functions** always **precede** their **arguments**. For example

```
max(x*x,x+3*y)
```

INFIX

corresponds to

```
(max (* x x) (+ x (* 3 y)))
```

- In Python: ['max', ['*', 'x', 'x'], ['+', 'x', ['*', '3', 'y']]]

Prefix Notation (2)

- With prefix notation it is easier to see the **correspondence between expressions and syntax trees**.
- Let us see it step by step

['max', ['*', 'x', 'x'], ['+', 'x', ['*', '3', 'y']]]

Legend:
- terminal
- function

Prefix Notation (3)

- Let us indent the expression: ['max', ['*', 'x', 'x'], ['+', 'x', ['*', '3', 'y']]]

```
['max',
        ['*', 'x', 'x'],
        ['+', 'x', ['*', '3', 'y']]]
```

- Let us indent more:

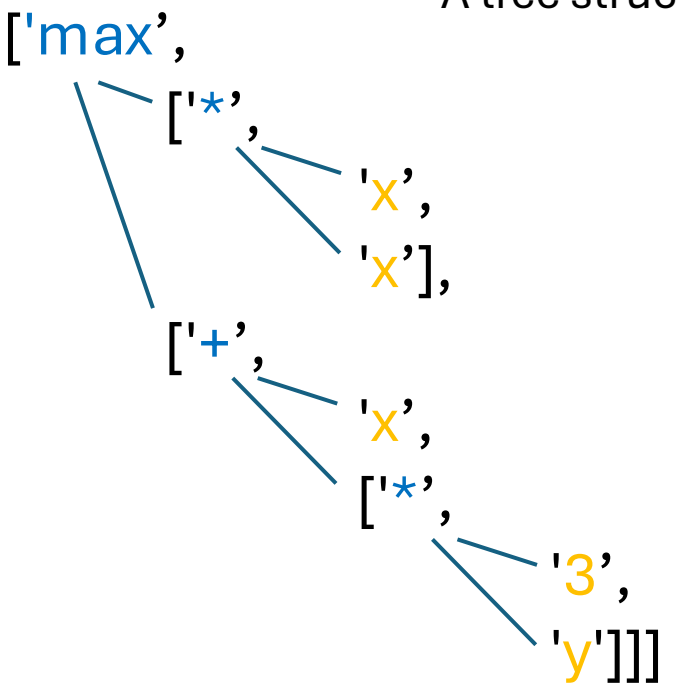
```
['max',
        ['*',
                'x',
                'x'],
        ['+',
                'x',
                ['*', '3', 'y']]]
```

Prefix Notation (4)

...and even more:

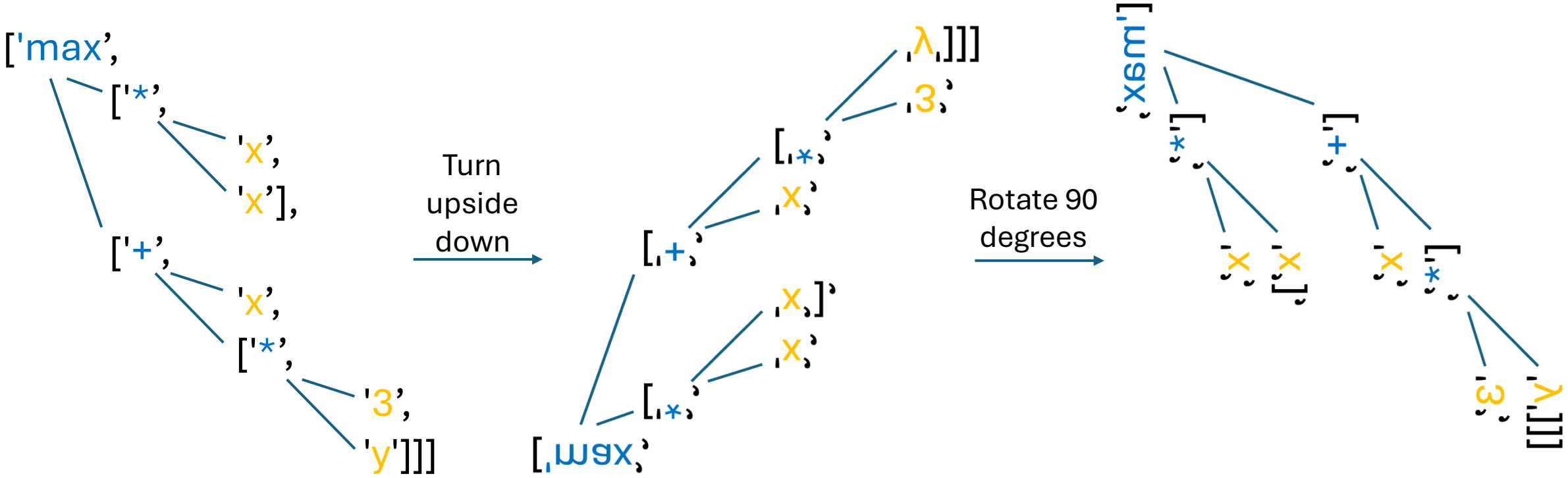
```
['max',
        ['*',
                'x',
                'x'],
        ['+',
                'x',
                ['*', '3', 'y']]]
```

A tree structure emerges

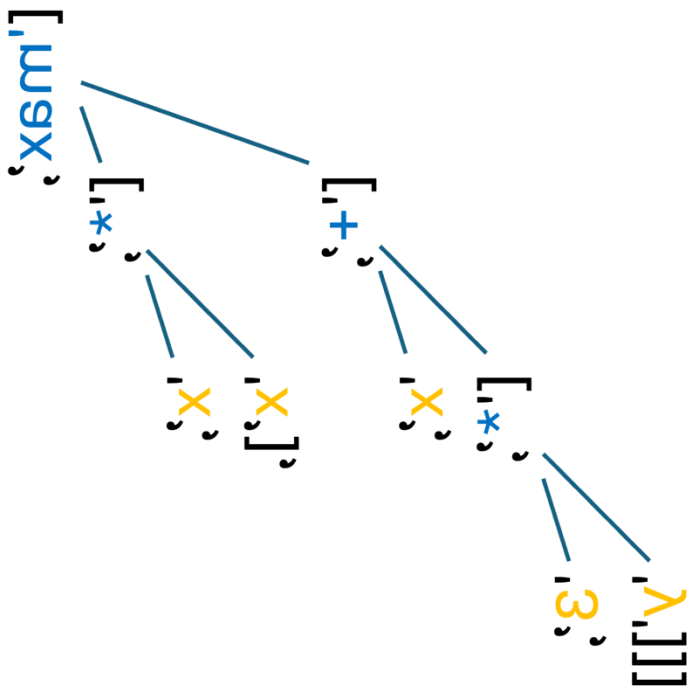


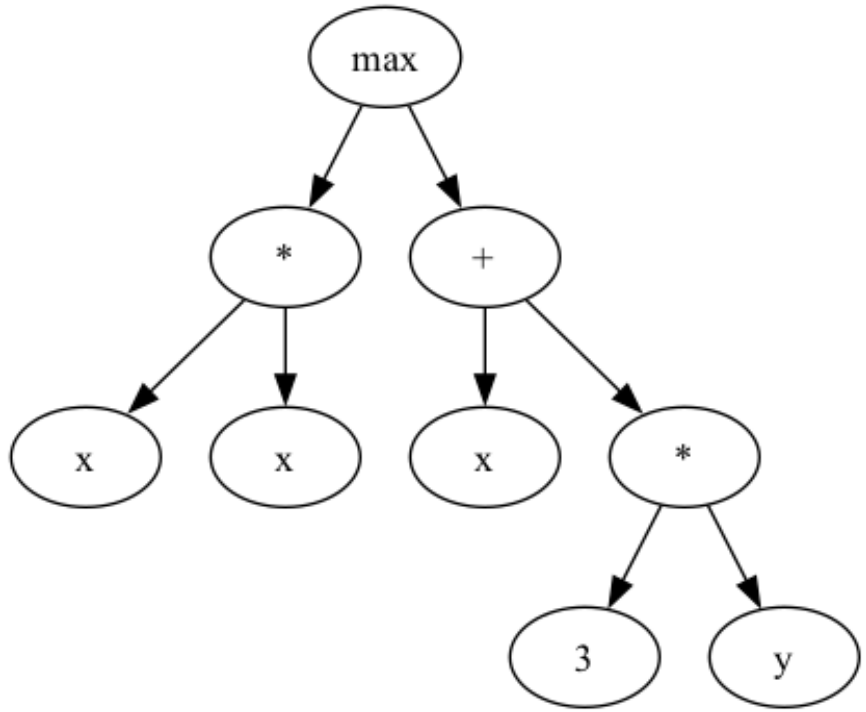
Prefix Notation (5)

...and finally...



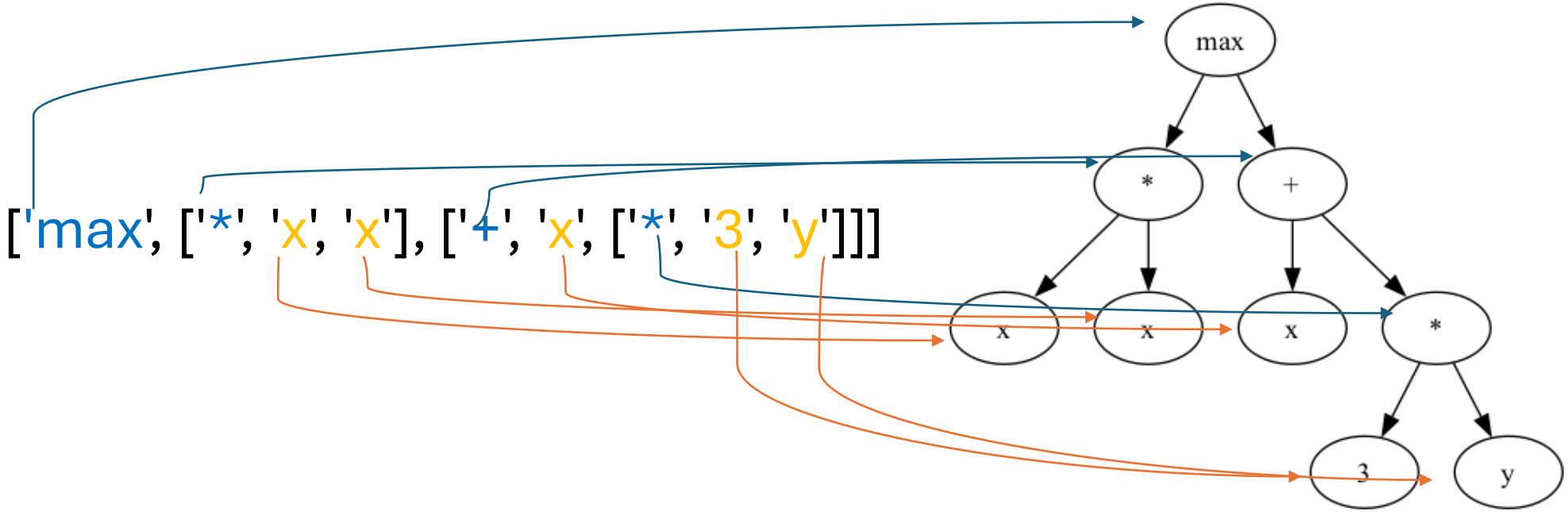
Prefix Notation (6)





Prefix Notation (7)

- With prefix notation it is easier to see the **correspondence between expressions and syntax trees**.



- **Recursive** functions can convert prefix into infix and vice versa.

Implementation Note

- In **Lisp**, **Python** and many other languages **lists are fundamental data types** and expressions are represented internally as trees. So, GP can directly be performed using lists.
- However:
  - **Tree/list-based representations of programs are memory-inefficient** (for each node we need to store content and pointers as its arguments/children and there may be other sources of memory waste).
  - **Program size is hard to work out** (requires recursion).

Prefix Notation (8)

- If all functions have a fixed arity (#arguments), the **brackets become redundant** in prefix-notation expressions.
- For example,

```
(max (* x x)(+ x (* 3 y)))
```

  is equivalent to

```
max * x x + x * 3 y
```

- In Python: ['max', '*', 'x', 'x', '+', 'x', '*', '3', 'y']
- So, often GP trees are stored internally as **linear sequences of instructions**.

Syntax Errors Are Impossible

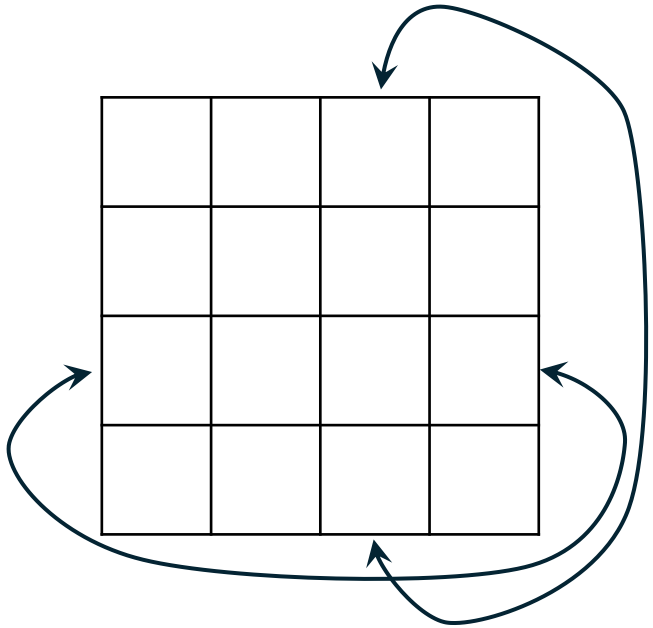
- **IF** all programs in the **initial population** of a run of GP are **syntactically valid**, executable programs,
- **AND** the **genetic operations** performed during the run are designed to produce offspring that are **syntactically valid**, executable programs,
- **THEN** **every individual created during a run of GP is a syntactically valid, executable program**.

Run-time Errors Can Be Avoided

- **IF** all functions in the primitive set can take as input the results produced by any other function or terminal (**closure**),
- **THEN** **run-time errors are avoided**.

Examples of closed primitive sets

- `{AND,OR,NOT, x, y, true}` where `x` and `y` are Boolean variables, and `true` is a Boolean constant.
- {`PROG2, IF_FOOD_AHEAD, TURN_LEFT, TURN_RIGHT, MOVE_FORWARD`} if world is ***toroidal***.



Examples of closed primitive sets

- `{+, -, *, x, y, 1, 0}` where `x` and `y` are **integer variables**: in theory there is no way of generating a non-valid expression but **integer overflow** may occur.
- `{+, -, sin, cos, exp, x, y}` where `x` and `y` are **floating-point variables**: in theory no problem because all functions accept arguments in (-∞,+ ∞) but **floating-point overflow** is very likely.

Examples of non-closed primitive sets

- `{+, -, *, /, x, y, 1, 0}` where `x` and `y` are **floating-point variables**: possible division by `0`, e.g. `x/0` or `(x+y)/(x-x)`
- `{+, -, sin, cos, log, x, y, 0, 1}` where `x` and `y` are **floating-point variables**: possible negative argument for `log`, e.g. `log(0-x)`
- `{+, *, sqrt, x}` can be non-closed depending on the range for `x`

Protected Functions

- Achieving closure requires modifying the primitive set appropriately, e.g. **using protected versions of division, logarithm, square root**, etc.

Function	*Protected version*	
`x/y`	`1`	if y = 0
	`x/y`	otherwise
`log(x)`	`0`	if x = 0
	`log(|x|)`	otherwise
`sqrt(x)`	`sqrt(|x|)`	

Exceptions

- An alternative to protected functions is to **trap FP exceptions** and strongly reduce the fitness of non-valid expressions when they cause math errors.
- However, with this method, if the likelihood of generating invalid expressions is very high, all individuals might get the same fitness and evolution **might not start**.

Primitive Set Sufficiency

- **Sufficiency** requires that the primitives in the primitive set are **capable of expressing the solutions to the problem**, i.e. that the set of all the possible recursive compositions of such primitives includes at least a solution.
- Sufficiency is **guaranteed only for some problems**, when theory (or experience with other methods) tells us that a solution can be obtained by combining the elements of the primitive set.

A Sufficient Primitive Set

- The primitive set $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOT}, x_1, x_2, \ldots, x_N\}$

  is **always sufficient** for Boolean function induction problems, since it can produce all Boolean functions of the variables x_1, x_2, ..., x_N.

Proof

- Any Boolean function **f** can be expressed with a **truth table**, which is also **equivalent to the** (**disjunctive normal form**, **DNF**) **formula**:

x_1	x_2	…	x_N	f
0	0	…	0	$a_{00...0}$
1	0	…	0	$a_{10...0}$
…	…	…	…	…
1	1	…	1	$a_{11...1}$

$$
\begin{aligned}
&\mathrm{OR}(\mathrm{AND}(a_{00...0}, \mathrm{NOT}(x_1), \mathrm{NOT}(x_2), ..., \mathrm{NOT}(x_N)),\\
&\quad \mathrm{AND}(a_{10...0}, x_1, \mathrm{NOT}(x_2), ..., \mathrm{NOT}(x_N)),\\
&\quad \ldots\\
&\quad \mathrm{AND}(a_{11...1}, x_1, x_2, ..., x_N)))
\end{aligned}
$$

An Insufficient Primitive Set (PS) (1)

- If PS is not sufficient for a particular application, GP can only **develop programs that approximate the desired one**.
- Let us consider the problem of finding a program that that implements **`exp(x)`** using the primitives in

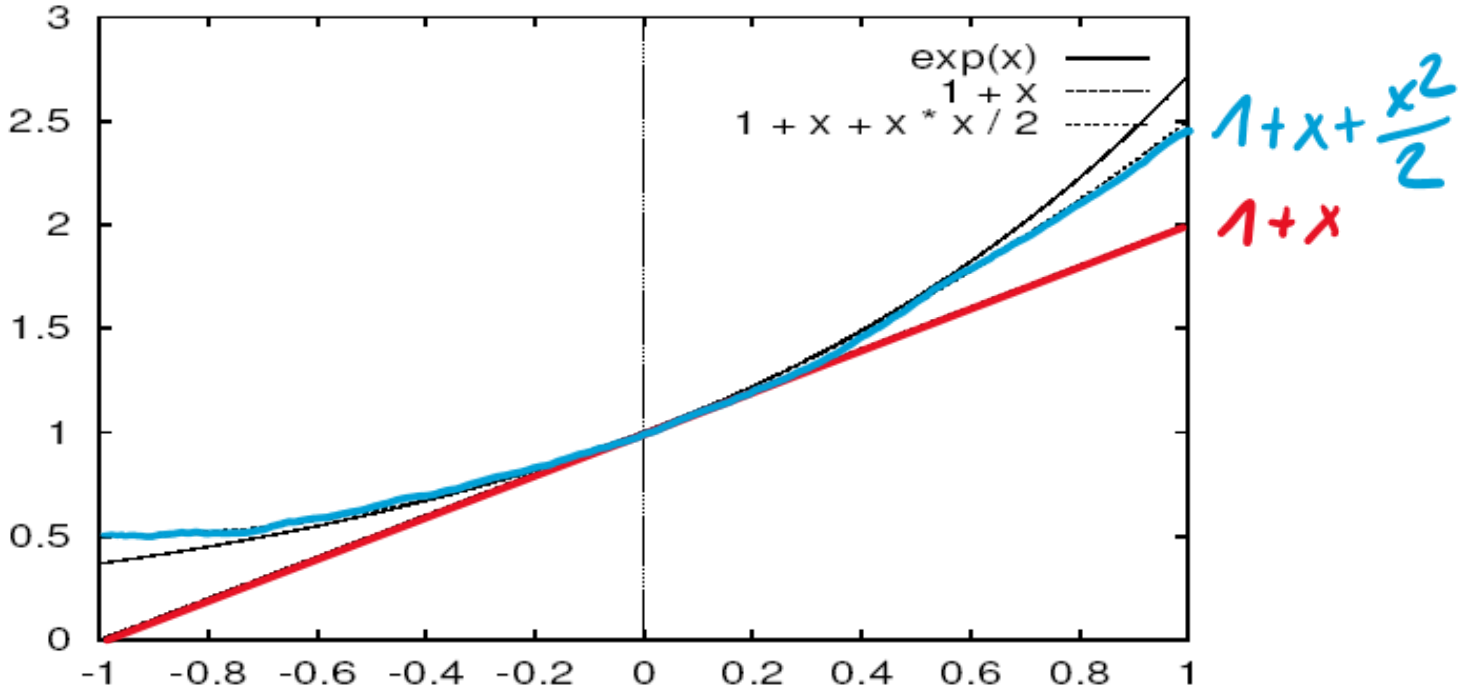
C = `{+, -, *, /, x, 0, 1, 2}`

- **`exp` cannot be expressed** in closed form using the elements of C
- Why? It is a transcendental function

Cannot be written down using algebra

An Insufficient Primitive Set (2)

- The typical behaviour of GP is to produce finite **algebraic approximations** of `exp(x)`, such as `1 + x` or `1 + x + x²/2`



Search Space

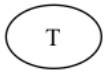
- The **theoretical search space of GP** is the **set of all the possible (recursive) compositions** of the elements of the primitive set C. This is infinite.
- The **real search space of GP is finite**. It includes "only" all the (recursive) compositions of the functions in C whose size (e.g. depth or number of nodes) does not exceed a **prefixed maximum value** (e.g. the memory available on the computer).
- Nonetheless, the search space is huge.

Size of GP Search Space (1)

- Let us compute the number, N(d), of different trees of maximum depth d present in the search space explored by GP when

$$C = \{\texttt{AND}, \texttt{OR}, \texttt{NOR}, \texttt{NAND}, \texttt{x1}, \texttt{x2}, \texttt{x3}\}$$

Size of GP Search Space (2)

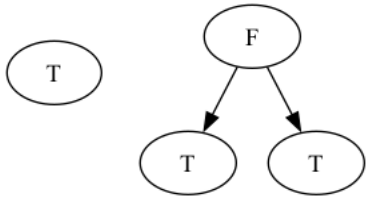
- T = terminals: 3 choices `x1`, `x2`, `x3`
- F = functions: 4 choices `AND`, `OR`, `NOR`, `NAND`
- If max depth $d = 0$
- Only 1 possible tree shape ➔ 

- Labelling with number of replacements ➔

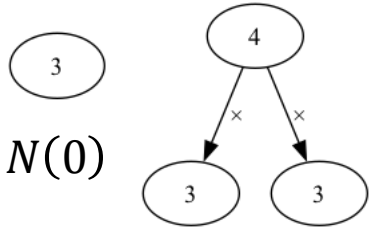
- So, $N(0) = 3$

Size of GP Search Space (3)

- If max depth $d = 1$
- Two possible tree shapes ➔



with replacements ➔

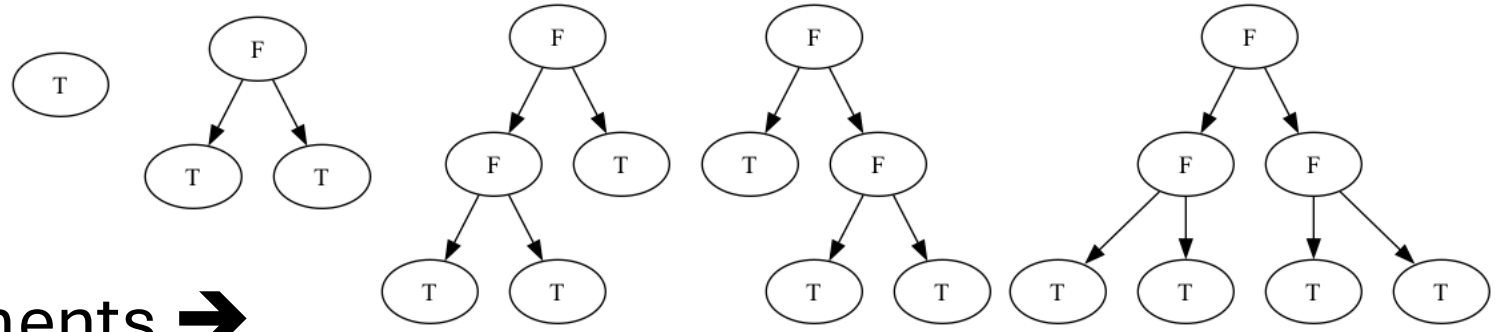


$N(0)$

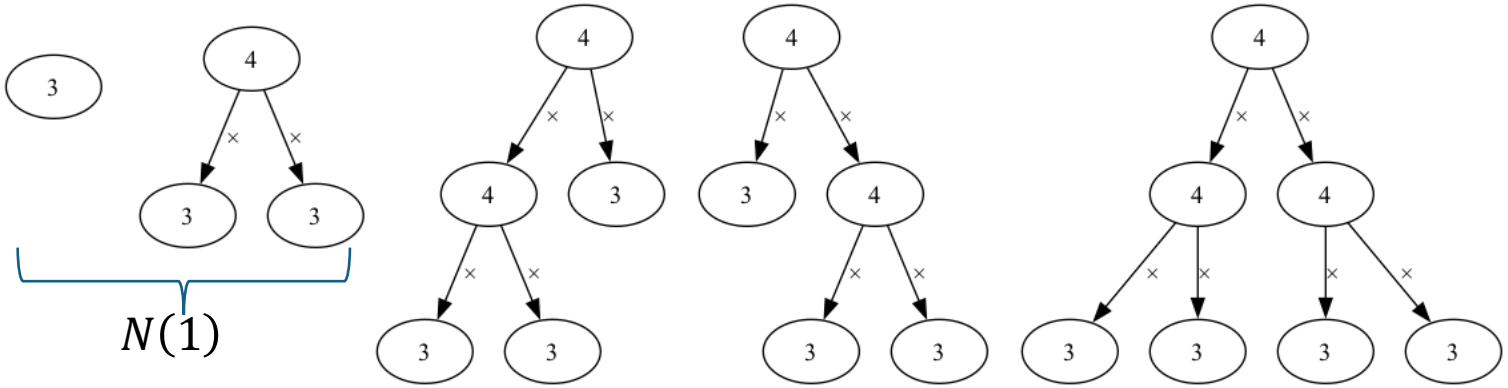
- So, $N(1) = N(0) + 4 \times 3 \times 3$ = 3 + 36 = **39**

Size of GP Search Space (4)

- If max depth $d = 2$
- Five possible tree shapes ➔



with replacements ➔



- So, $N(2) = N(1) + 4\times4\times3\times3\times3 + 4\times4\times3\times3\times3 + 4\times4\times4\times3\times3\times3\times3$
  $= 39 + 432 + 432 + 5184 =$ **6087**

Bigger depth?

$N(0) = 3$

$N(1) = 39$

$N(2) = 6087$

$N(6) = 3.8 \times 10^{69}$

$N(17) = 1.1 \times 10^{143735}$

General Answer

- Assuming $arity(p)$ returns the number of arguments of primitive p and that $arity(T) = 0$ (note $n^0 = 1$ for any n)
- Recursion:

$$N(d) = N(d-1)^{\mathtt{arity(p_1)}} + N(d-1)^{\mathtt{arity(p_2)}} + \ldots + N(d-1)^{\mathtt{arity(p_n)}}$$

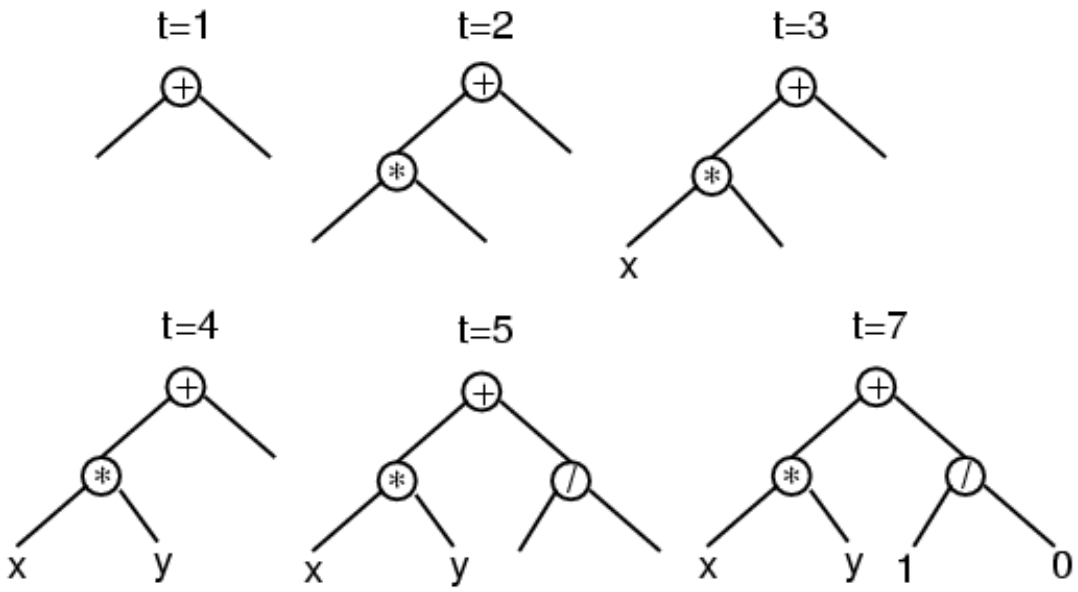
where $N(0)$ = number of terminals

“Full” Initialisation Method

- Nodes are taken from the function set until a **maximum tree depth** is reached. Beyond that depth only terminals can be chosen.

Example “Full”

- Assume max depth d = 2.
- At time t=1 a “+” node is selected as the root of the tree.
- At t=2 the 1st child of the root (“+”) is chosen. It is a product, “*”.
- At t=3 the 1st child of the “*” is chosen. We have reached max depth, so we pick a leaf, “x”.
- At t=4 the 2nd child of the “*” is chosen. Again it is a leaf, “y”.
- At t=5 the 2nd child of the root (“+”) is chosen. It is a division, “/”.
- At t=6 and t=7 we complete the tree with two more leaves.

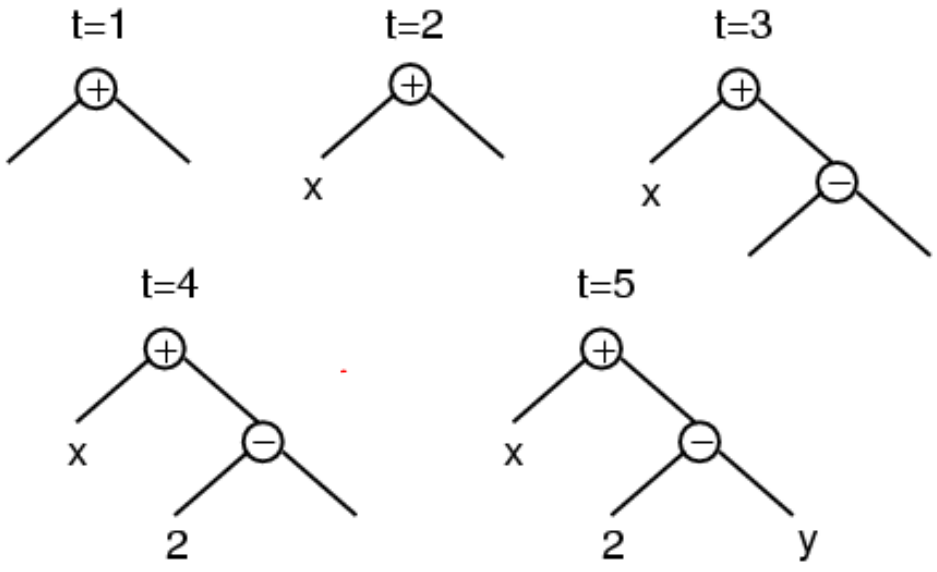


“Grow” Initialisation Method

- It behaves like “full” except it allows the selection of nodes from the whole primitive set until the depth limit is reached.

Example “Grow”

- Again, assume max depth d = 2.
- At time t=1 a “+” node is selected as the root of the tree.
- At t=2 the 1st child of the root (“+”) is chosen. It can be either a function or a terminal. It is the terminal, “x”.
- At t=3 the 2nd child of the root is chosen similarly. It is a subtraction, “-”.
- At t=4 the 1st child of the “-” is chosen. We have reached max depth, so we must pick a terminal. “2” is chosen.
- At t=5 the 2nd child of the “-” is chosen. Again it must be a leaf, “y”.



Implementation (1)

- If trees are represented in **prefix notation**, initialisation with the "full" and the "grow" methods can be performed using recursive procedures.
- Set up:

```
# Setup primitives
func_set = [['sin', 1], ['cos', 1], # Format: [Name, Arity]
            ['add', 2], ['sub', 2], ['mul', 2]]

term_set = ['x', '1', '0'] # Format: Name
```

Implementation (2)

- **“Full” method**:

```python
import random

def full(maxD):
    # At max depth, pick a terminal
    if maxD == 0:
        return random.choice(term_set)
    else:
        # Pick a function and fill its arguments
        func, arity = random.choice(func_set)
        expr = [func]
        for _ in range(arity):
            expr.append(full(maxD - 1))
        return expr

print(full(2))
```

Implementation (3)

- **“Grow” method**:

```
import random

prob_term = len(term_set) / (len(term_set) + len(func_set))

def grow(maxD):
    # At max depth OR randomly decided to insert a terminal
    if maxD == 0 or random.random() < prob_term:
        return random.choice(term_set)
    else:
        # Pick a function and fill its arguments
        func, arity = random.choice(func_set)
        expr = [func]
        for _ in range(arity):
            expr.append(grow(maxD - 1))
        return expr

print(grow(2))
```

ONLY DIFFERENCE!

Ramped “half-and-half” Initialisation

- This method combines the previous two methods to get a higher diversity in the initial population.
- Fore example: If P = 1000, d_{min}= 1 and d_{max}= 5, for each possible depth (i.e. d =1, 2, 3, 4 and 5), 100 individuals will be produced with the “grow” method and 100 with the “full” method.

Simplest (approx.) implementation

```
def ramped(minD, maxD):
    maxD = random.randint(minD, maxD) # includes end points
    method = random.choice(['grow','full'])
    if method == 'grow':
        return grow(maxD)
    else:
        return full(maxD)

for _ in range(5):
    print(ramped(0,3))
```

Execution Strategies

- **Full off-line compilation** and linking of the individuals produced in each generation.
- **Full on-line compilation** and dynamic linking of each individual generated.
- **Special-purpose on-line compilation** of the primitive set into machine code
- **Virtual-machine code** compilation
- **Interpretation**

Compilation vs. interpretation

- Compiling programs before executing them takes time.
- Executing compiled programs is quicker.
- Compilation is convenient when:

$$\tau_c + N \times t_c < N \times t_i$$

τ_c	compilation time
N	number of executions (fitness cases)
t_i	execution time when interpreted
t_c	execution time when compiled

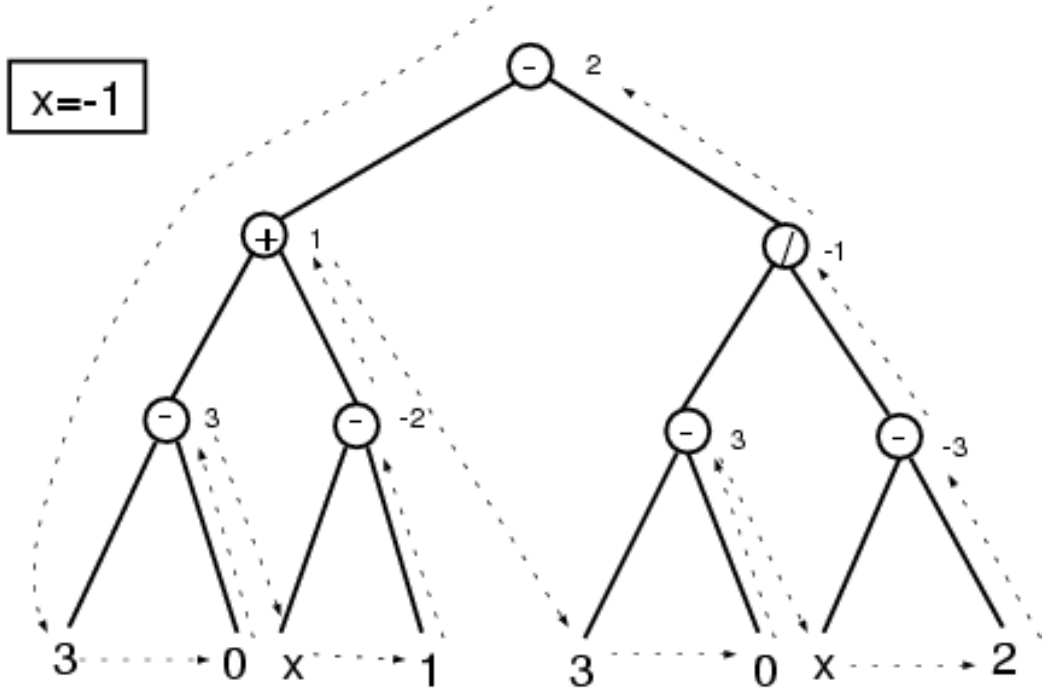
Interpretation (1)

- Some languages like Lisp and Python have **built-in interpreters**. In such languages interpreting/executing programs is trivial.
- For other languages it is necessary to **build an interpreter**.
- A full interpreter is seldom needed, since
  - Programs are not represented as lines of text, so a lexical analysis is unnecessary.
  - Not all the primitives of the language are used.

Program Interpretation

- Interpreting a program tree means recursively traversing the tree (in “depth-first” order) and executing its nodes only when their arguments are known.

  E.g.



Recursive Interpreter for GP

```python
# Setup primitives
from operator import add, sub, mul
func_set = [['add', 2], ['sub', 2], ['mul', 2]]
term_set = ['x', '1', '0', '-1'] # Format: Name

def interpreter(program, x):
    if program in term_set: # just one terminal
        # turn string into value
        return eval(program)
    else: # Larger program
        # find out function to invoke and its arguments
        func, arguments = program[0], program[1:]
        # Do recursion on arguments to get their value
        evaluated_arguments = [interpreter(arg, x) for arg in arguments]
        # Find implementation of function and call it with evaluated args
        evaluated_func = eval(func)
        return evaluated_func(*evaluated_arguments)

prog = full(2)
for x in range(5):
    result = interpreter(prog, x)
    print(prog,'x =',x,'-->',result)
```

```
['sub', ['add', 'x', 'x'], ['sub', '0', '-1']] x = 0 --> -1
['sub', ['add', 'x', 'x'], ['sub', '0', '-1']] x = 1 --> 1
['sub', ['add', 'x', 'x'], ['sub', '0', '-1']] x = 2 --> 3
['sub', ['add', 'x', 'x'], ['sub', '0', '-1']] x = 3 --> 5
['sub', ['add', 'x', 'x'], ['sub', '0', '-1']] x = 4 --> 7
```

Problems with “non-functional” instructions (1)

- Some instructions like **conditionals** or **loops** cannot be implemented properly as normal functions.
- For example, when interpreting

  `IF(<condition>, <do_this>, <do_that>)`

  `execute` first evaluates all the arguments and then it invokes `IF`, which returns the value of `<do_this>` or `<do_that>` depending on the value of `<condition>`.

Problems with “non-functional” instructions (2)

- When there are no functions or terminals with side effects this is only a **waste of computation**.
- However, the interpretation of an expressions like

  `IF FOOD_ON_THE_LEFT, TURN_LEFT, TURN_RIGHT`

  would produce the **wrong behaviour**. Same with:

  `IF <cond>, mem_set(val), mem_get()`

  where `mem_set(val)` and `mem_get()` are memory ops
- Another example: `REPEAT <times>, <something>`

  have **no way of executing `<something>` in different contexts**.

Macros in GP (1)

- The solution to this problem is to use **special functions**, called **macros** in the GP literature, which receive as input their **arguments unevaluated** (i.e. as sub-expressions).
- Macros are free to decide if, when and how many times to evaluate their arguments by **explicitly calling the interpreter**.

Macros in GP (2)

- For example, a macro implementation for **IF** is:

```
def if_else(cond, then_prog, else_prog, x):
    evalulated_cond = interpreter(cond,x)
    if evalulated_cond > 0:
        return interpreter(then_prog, x)
    else:
        return interpreter(else_prog, x)
```

Compare with:

```
def if_else(cond, then_prog, else_prog):
    if cond > 0:
        return then_prog
    else:
        return else_prog
```

Handling Macros in eval

- In order to handle macros properly, the interpreter has to be able to **distinguish between macros and normal functions**.

```
# Setup primitives
from operator import add, sub, mul
func_set = [['add', 2], ['mul', 2], ['if_else', 3]]
term_set = ['x', '1', '-1'] # Format: Name
macros = ['if_else']
```

New interpreter for GP

```python
def interpreter(program, x):
    if program in term_set: # just one terminal
        # turn string into value
        return eval(program)
    else: # Larger program
        # find out function/macro and its arguments
        func, arguments = program[0], program[1:]
        if func in macros: # Macro
            # Find implementation of function and call it with UNEVALUATED args
            arguments.append(x) # Macros need to know x
            evaluated_func = eval(func)
            return evaluated_func(*arguments)
        else: # Function
            # Do recursion on arguments to get their value
            evaluated_arguments = [interpreter(arg, x) for arg in arguments]
            # Find implementation of function and call it with evaluated args
            evaluated_func = eval(func)
            return evaluated_func(*evaluated_arguments)
```

Running the interpreter

```
prog = ['if_else', ['add', 'x', '-1'], ['add', '1', 'x'], '1']
for x in [0, 1, 2]:
    result = interpreter(prog, x)
    print(prog,'x =',x,'-->',result)
```

```
['if_else', ['add', 'x', '-1'], ['add', '1', 'x'], '1'] x = 0 --> 1
['if_else', ['add', 'x', '-1'], ['add', '1', 'x'], '1'] x = 1 --> 1
['if_else', ['add', 'x', '-1'], ['add', '1', 'x'], '1'] x = 2 --> 3
```

Mutation Implementation (1)

- The implementation of mutation is representation dependent.
- For GP trees represented as list of lists one needs to *recursively* identify a random sub-list in the parent and replace it with a randomly generate small program.
- The operations are performed on a copy of the parent so as not to destroy the original (as it could be used to produce more than one offspring and/or be cloned)

Mutation Implementation (2)

```python
def mutate(prog): # NOTE: Not necessarily best choices of constants!
    if isinstance(prog, list):
        mut_point = random.choice(range(1, len(prog)))
        if random.random() < 0.5:
            prog[mut_point] = grow(2)
        else:
            prog[mut_point] = mutate(prog[mut_point])
    else:
        prog = grow(1)
    return prog

prog = full(2)
print('PARENT', prog)
print('OFFSPRING')
for _ in range(5):
    offspring = mutate(prog[:])
    print(offspring)
```

```
PARENT ['if_else', ['mul', '1', 'x'], ['mul', '1', 'x'], ['mul', 'x', '1']]
OFFSPRING
['if_else', ['mul', '1', 'x'], ['mul', '1', 'x'], '1']
['if_else', ['mul', '1', 'x'], ['mul', '1', 'x'], '1']
['if_else', ['mul', '1', 'x'], ['mul', '1', '-1'], ['mul', 'x', '1']]
['if_else', ['mul', '1', 'x'], ['mul', '1', '-1'], ['mul', 'x', ['add', '1', '-1']]]
['if_else', 'x', ['mul', '1', '-1'], ['mul', 'x', ['add', '1', '-1']]]
```

Crossover Implementation (1)

- The implementation of crossover is representation dependent.
- For GP trees represented as list of lists one needs to *recursively* identify a random sub-list in the first parent and replace it with a *recursively randomly selected sub-list from the second parent*.
- The operations are also performed on copies.

Crossover Implementation (2)

- Identification of a random sub-list/sub-tree in a program

```
import random
from copy import deepcopy

def get_random_subtree(prog):
    if not isinstance(prog, list) or random.random() < 0.3:
        # Return the current node (terminal or the whole subtree)
        return deepcopy(prog)
    else:
        # Randomly decide to go deeper into the tree
        idx = random.choice(range(1, len(prog)))
        return get_random_subtree(prog[idx])
```

Crossover Implementation (3)

```python
def crossover(parent1, parent2):
    # We work on copies to avoid modifying the original population
    offspring = deepcopy(parent1)

    # If the offspring is a list, we can pick a sub-branch to replace
    if isinstance(offspring, list):
        # Pick a random point in parent 1 (skipping the operator at index 0)
        idx = random.choice(range(1, len(offspring)))

        if random.random() < 0.5:
            replacement_subtree = get_random_subtree(parent2)
        else:
            replacement_subtree = crossover(offspring[idx], parent2)

        # Perform the swap
        offspring[idx] = replacement_subtree
        return offspring
    else:
        # If parent1 is just a terminal, crossover might just return offspring
        return offspring
```

Crossover Implementation (4)

- Testing the code

```
p1 = full(2)
p2 = full(2)
print('parent 1', p1)
print('parent 2', p2)
for _ in range(5):
    print('offspring',crossover(p1,p2))
```

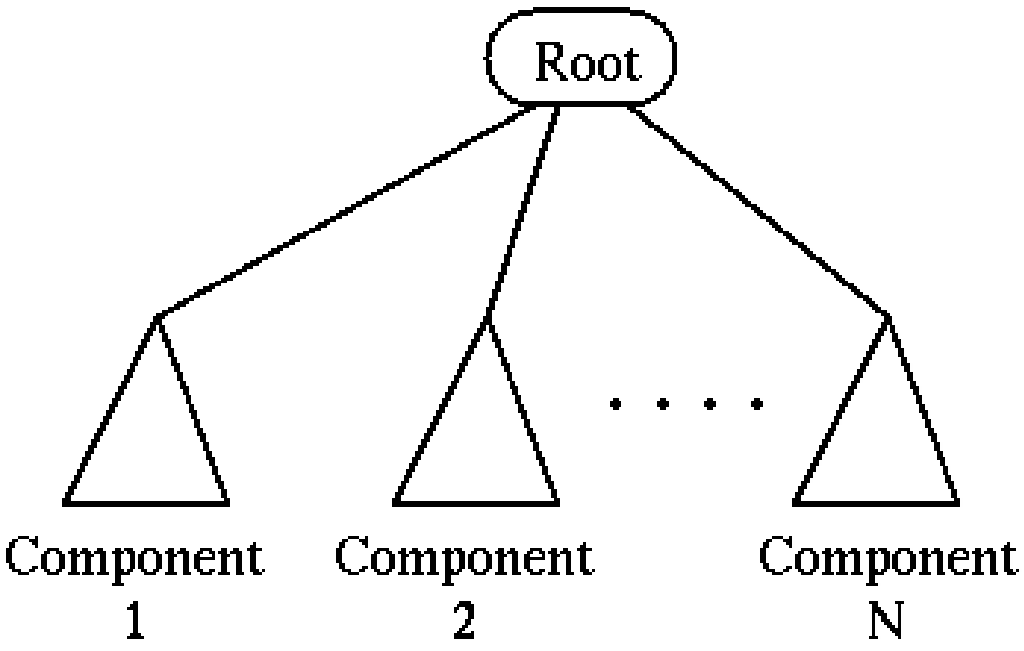
```
parent 1 ['add', ['add', '-1', 'x'], ['add', 'x', 'x']]
parent 2 ['mul', ['if_else', '-1', '1', '1'], ['mul', 'x', 'x']]
offspring ['add', ['add', 'x', 'x'], ['add', 'x', 'x']]
offspring ['add', ['add', '-1', 'x'], ['mul', ['if_else', '-1', '1', '1'], ['mul', 'x', 'x']]]
offspring ['add', ['add', '-1', 'x'], ['mul', ['if_else', '-1', '1', '1'], ['mul', 'x', 'x']]]
offspring ['add', ['add', ['if_else', '-1', '1', '1'], 'x'], ['add', 'x', 'x']]
offspring ['add', ['add', '-1', 'x'], ['add', ['if_else', '-1', '1', '1'], 'x']]
```

Multi-tree Representation

- GP **programs** can be composed of **multiple trees** representing different **components** of the program
- We can represent a multi-tree program as an **array/list of trees**.
- But **multi-tree programs are often represented as trees** with a **special root node** with as many children as there are components in the program.
- The root node acts as glue for the multiple trees, it is not meant to be executed.
- The components are also called **branches**.
- **Specialised initialization, operators and interpreters** are required.

Example

- Here, a special node called “Root”, has N children.
- Each child is a tree which represent a different component of a program.
- **Each component may play a different role** in the program.
- For example, if we want to evolve programs to control a robotic football team, Component 1 might represent the program controling the goalkeeper, Component 2 may control a defender, etc.



Divide and Conquer (1)

- In the presence of a complex problem, every good engineer will try and use the following ”**divide-and-conquer"** strategy:
  1. **Decompose** the problem into sub-problems which are simpler to solve
  2. **Solve** the sub-problems
  3. **Combine** the solutions of the sub-problems to build the solution for the original problem.
  4. **Iterate** 1, 2 and 3 for the sub-problems if they are still too difficult.

Divide and Conquer (2)

- This strategy applies also to programming. In this case the **solutions of the sub-problems** are usually **subroutines**.
- Good programmers try to build **parameterised subroutines** which solve entire classes of sub-problems instead of a single one.
- Parameterised reuse **improves the quality** (e.g. generality) of programs and reduces their complexity.

Automatically Defined Function (ADF)

- It is possible to generalise GP to exploit the ``divide-and-conquer'' strategy, i.e. to **discover and use automatically good reusable subroutines**.
- This can be easily obtained by:
  - Using a **multi-tree program representation** with a fixed number of branches (components)
  - Interpreting some of the branches as **Automatically Defined Functions** (ADFs), i.e. subroutines
  - Interpreting one of the branches as the **main program**.

ADF Example

Here, the node called “Root” has 2 children.

The first child is the **function-defining branch,** which represents the program (omitting quotes):

`[+, [-, arg1, arg2], arg3]`

where `arg1`, `arg2` and `arg3` are the *dummy arguments* of the function defined by this branch.

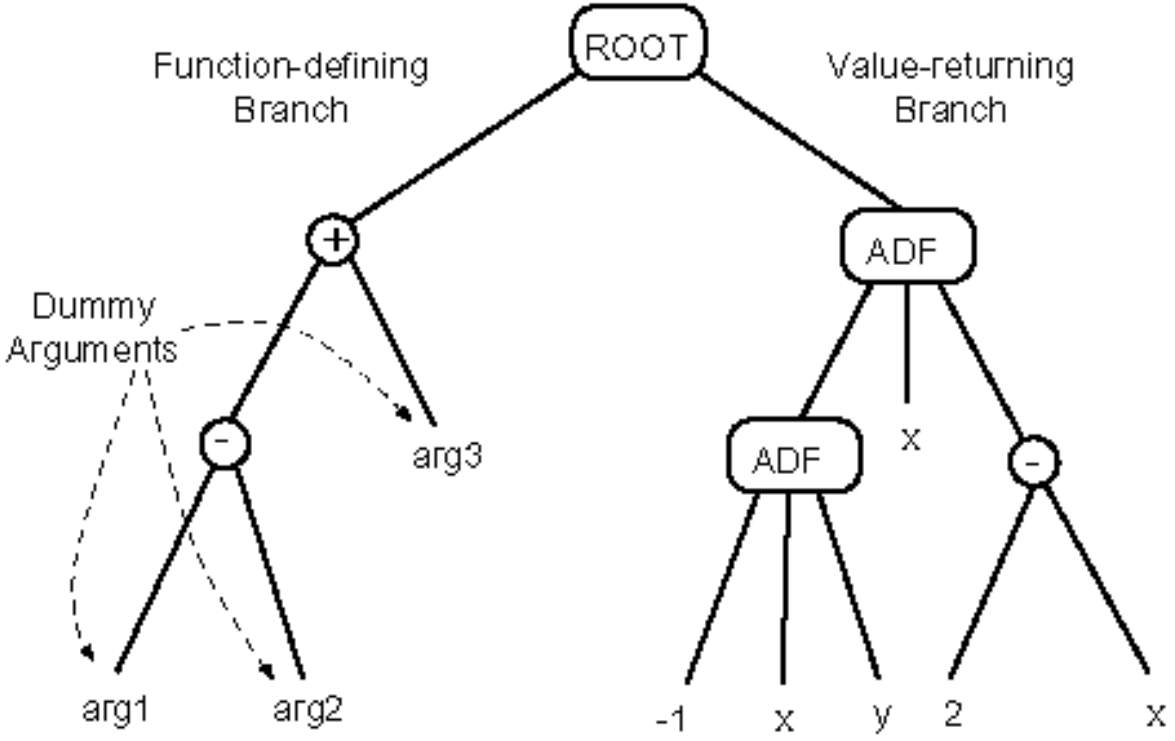
The second child is the **value-returning branch**, i.e., the main for the program

This can (and does) use the automatically defined function (ADF) described by the first branch.

The main program is

`[ADF, [ADF, -1, x, y], x, [-, 2, x]]`

Note: **the two branches have different primitive sets**.



Changes required by ADFs (1)

- Different branches have **different function and terminals sets**:
  - The **main program cannot have dummy arguments** but can use ADFs
  - **ADFs should not use global variables**, i.e. variables not passed as arguments (better reusability, no side-effects,…)
  - To **prevent infinite recursion**, ADFs should not call themselves nor other ADFs that in turn can call them.
  - ADFs can use safely the ADFs defined “on their left”.

Changes required by ADFs (2)

- The **random initialisation** procedure must generate trees with the **root special node** as root node and must use **different terminals and functions** for the different children of the root node.
- **Crossover** has to be slightly modified to ensure it swaps sub-trees taken from the **same branches** of the parents.
- **Mutation** requires generating random trees using of the same primitive set as the subtree to be excised.

Example: even-4 parity function

- The **even-4 parity function** is a Boolean function of 4 variables which returns 1 (or true) if an even number of its arguments has value 1. It returns 0 (or false) otherwise.
- Problem: find a program that implements such a function using `AND`, `OR`, `NAND` and `NOR` functions.
- Again this can be seen as a **symbolic regression problem** with the following dataset:

```
{(x1i, x2i, x3i, x4i, yi)} =
   {(0,0,0,0,1)(0,0,0,1,0)(0,0,1,0,0)(0,0,1,1,1) .... }
```

Without ADFs

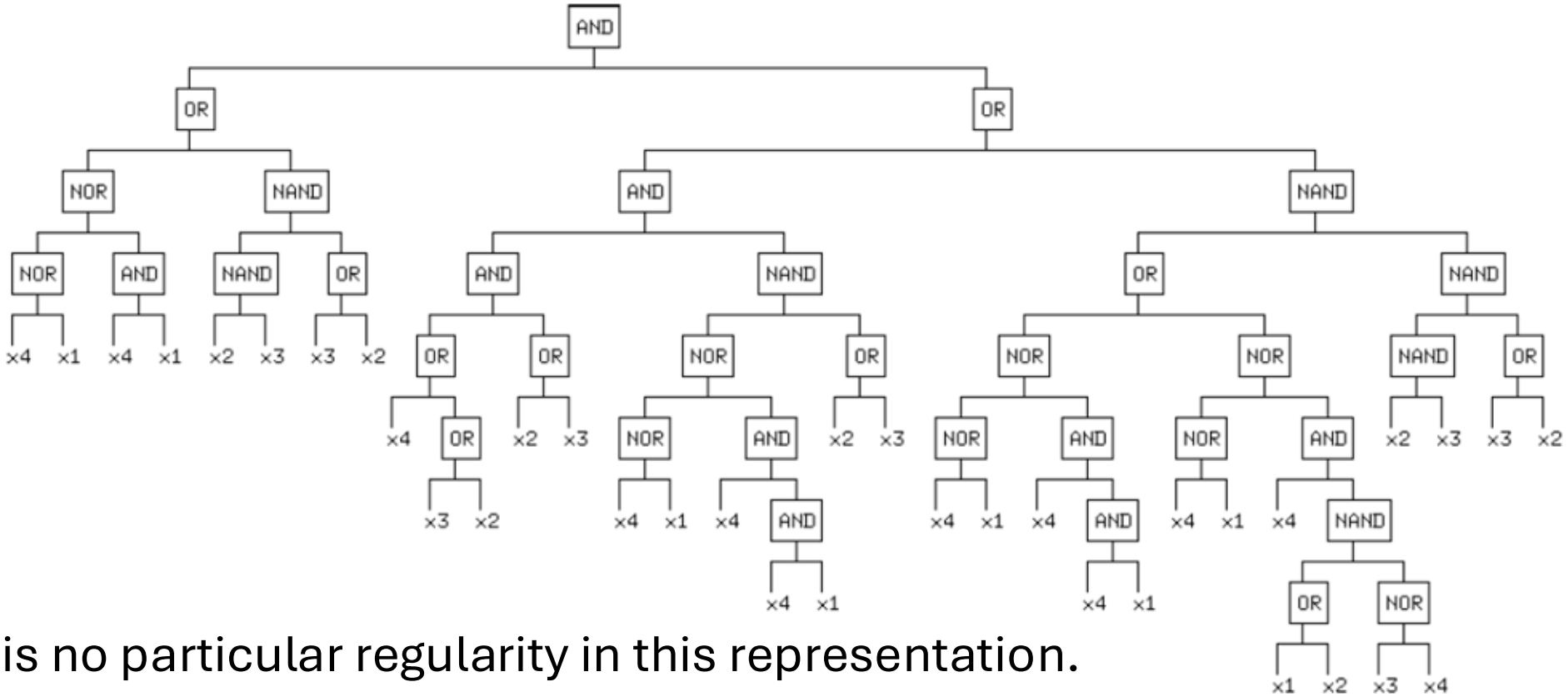
- GP parameters:
- Population size = 4000
- Function set = {AND OR NAND NOR}
- Terminal set = {x1 x2 x3 x4}
- Initial max depth = 4
- Initialisation method = Full
- Number of generations = 50
- Crossover probability = 0.7
- Mutation probability = 0
- Fitness = - Sum over i of abs(yi - eval(prog,x1i,x2i,x3i,x4i))

Solution

- After 31 generations, GP found the following error-free solution with 71 nodes:

```
[AND [OR [NOR [NOR x4 x1] [AND x4 x1]]
         [NAND [NAND x2 x3] [OR x3 x2]]]
     [OR [AND [AND [OR x4 [OR x3 x2]]
                   [OR x2 x3]]
              [NAND [NOR [NOR x4 x1]
                         [AND x4 [AND x4 x1]]]
                    [OR x2 x3]]]
         [NAND [OR [NOR [NOR x4 x1]
                        [AND x4 [AND x4 x1]]]
                   [NOR [NOR x4 x1]
                        [AND x4
                             [NAND [OR x1 x2]
                                   [NOR x3 x4]]]]]
              [NAND [NAND x2 x3] [OR x3 x2]]]]]
```

Its tree-based representation



- There is no particular regularity in this representation.
- The tree looks rather bushy.
- Its maximum depth is 9, but most leaves are at depths 5, 6, and 7.

With 2 ADFs – GP Parameters

General:

- Population size = 4000
- Number of generations = 50
- Crossover probability = 0.7
- Mutation probability = 0
- Fitness = same as before

ADF1:

- function set = {AND OR NAND NOR }
- terminal set = {arg1 arg2}
- initial max depth = 3
- initialisation method = Full

ADF2:

- function set = {AND OR NAND NOR ADF1}
- terminal set = {arg1 arg2}
- initial max depth = 3
- initialisation method = Full

Value-returning branch:

- function set = {AND OR NAND NOR ADF1 ADF2}
- terminal set = {x1 x2 x3 x4}
- initial max depth = 4
- initialisation method = Full

With 2 ADFs Solution

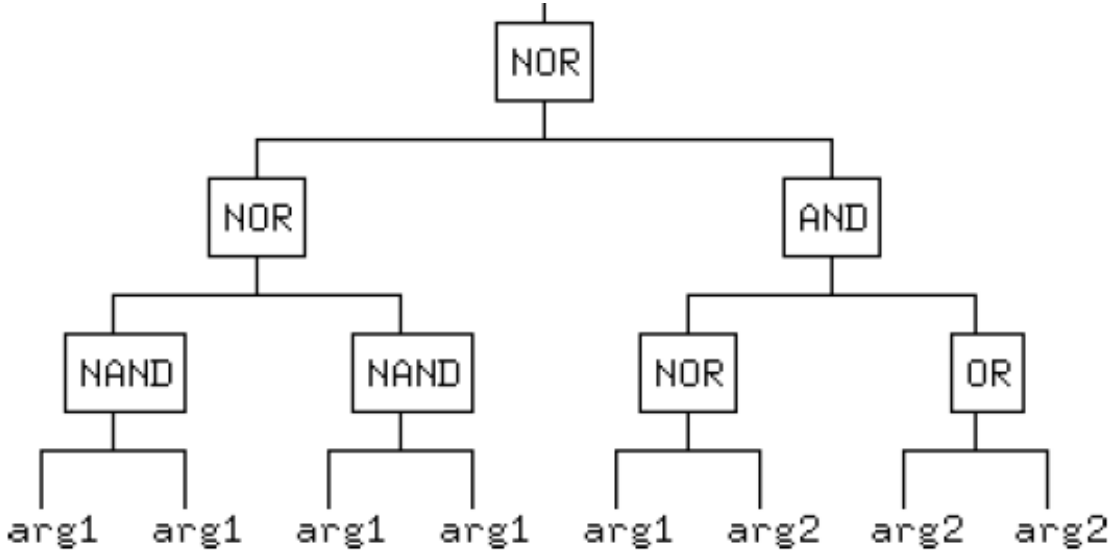
- After 13 generations, GP found the following error-free solution with 64 nodes:

```
[root [NOR [NOR [NAND arg1 arg1] [NAND arg1 arg1]]
           [AND [NOR arg1 arg2] [OR arg2 arg2]]]
      [NAND [NAND [AND arg1 arg2] [AND arg1 arg1]]
            [NAND [ADF1 arg2 arg2] [NAND arg1 arg1]]]
      [ADF1 [NAND [ADF1 [AND [NOR x1 x3] x1]
                        [OR x3 x2]]
                  [ADF2 [ADF2 x4 x3] [ADF2 x1 x2]]]
            [OR [ADF2 [ADF1 x3 x2] [ADF2 x1 x2]]
                [ADF1 [ADF2 x3 x4] [NOR x1 x3]]]]]
```

ADF1

ADF1 is the program:

```
[NOR [NOR [NAND arg1 arg1]
          [NAND arg1 arg1]]
     [AND [NOR arg1 arg2]
          [OR arg2 arg2]]]
```

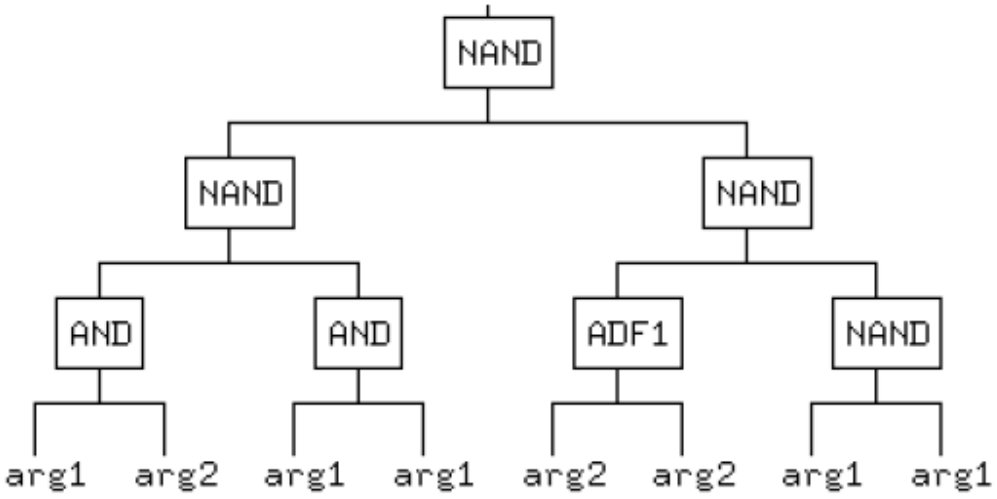


Graphically, it is a tree with 4 levels, all the leaves being at the same level (full tree).

ADF2

ADF2 is the program:

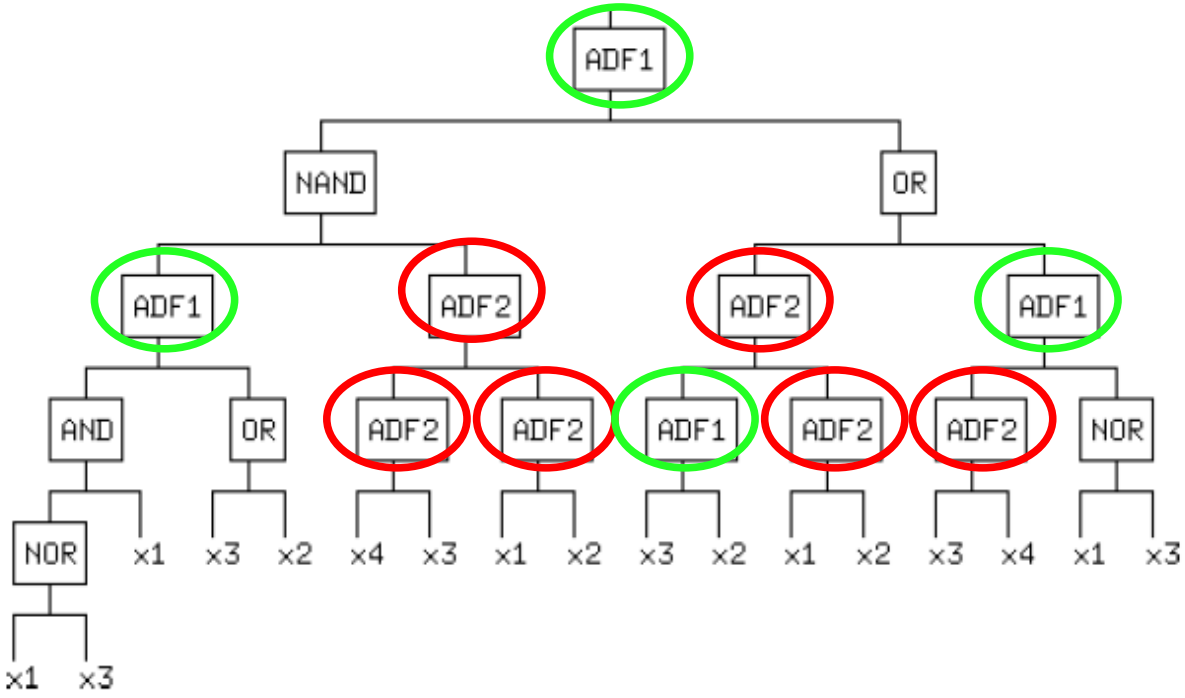
```
[NAND [NAND [AND arg1 arg2]
            [AND arg1 arg1]]
      [NAND [ADF1 arg2 arg2]
            [NAND arg1 arg1]]]
```



- Graphically, it is also a tree with 4 levels, with all the leaves at the same level (full tree).
- NOTE: ADF2 makes use of ADF1 once.

Main Program

```
[ADF1 [NAND [ADF1 [AND [NOR x1 x3]
                        x1]
                   [OR x3 x2]]
              [ADF2 [ADF2 x4 x3]
                    [ADF2 x1 x2]]]
        [OR [ADF2 [ADF1 x3 x2]
                  [ADF2 x1 x2]]
            [ADF1 [ADF2 x3 x4]
                  [NOR x1 x3]]]]]
```



- Graphically, it is a tree with 5 levels, with most of its leaves at level 4. It uses ADF1 **four** times and ADF2 **six** times.

Advantages and Disadvantages (1)

- For this problem the **structural complexity with ADFs** is only slightly smaller than without ADFs. However, in some more complex problems the saving may be much bigger.
- Reduction in complexity may be accompanied by a **reduction in computation load**.
- Programs with ADFs may **be easier to understand**.

Advantages and Disadvantages (2)

- For example, by building the truth tables of ADF1 and ADF2 we discover that

```
ADF1(arg1,arg2) = NOT arg1
ADF2(arg1,arg2) = NOT ( arg1 XOR arg2 )
                = EVEN2(arg1,arg2)
```

- So, we can rewrite the main program as

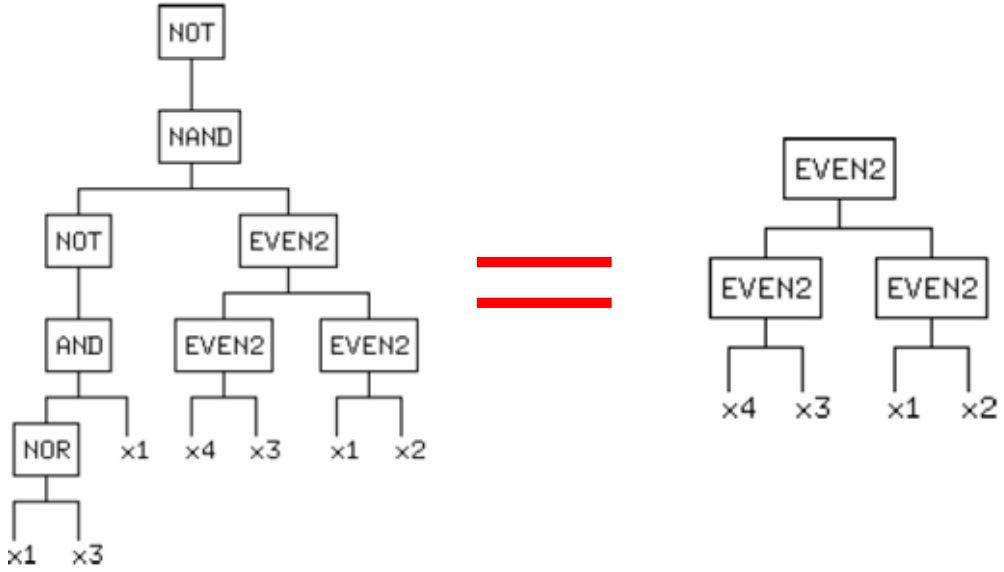
```
[NOT [NAND [NOT [AND [NOR x1 x3] x1]]
           [EVEN2 [EVEN2 x4 x3]
                  [EVEN2 x1 x2]]]]
```

- Further analysis shows that this is equivalent to

```
[EVEN2 [EVEN2 x4 x3]
       [EVEN2 x1 x2]]]]
```

Advantages and Disadvantages (3)

Graphically



- In problems which present either **minor internal regularities** or **regularities that cannot be easily captured by ADFs**, **ADFs can slow down GP** rather than help it.

What did we learn? (1)

- We have learned that GP trees can be represented as expressions in **prefix notation**
- We have learned that we can create syntactically correct programs, and we can avoid run-time errors when using **closed** primitive sets
- **Sufficient** primitive sets allow the GP to create programs that are capable of expressing the solution. **Insufficient** primitive sets only find **approximations**.

What did we learn? (2)

- We have seen how **huge** the **search space** is, even with small trees.
- The **Full** initialisation method takes nodes from the **function set** until the **maximum tree depth** is reached. The **Grow** initialisation method selects nodes from the whole **primitive set**. In practice, a **ramped half-and-half** initialisation is usually used.

What did we learn? (3)

- We have learned that different program execution strategies exist and that the decision "**compilation** vs. **interpretation**" in terms of computational complexity depends on the specific problem.
- We focused on interpretation and have seen that interpreting a **program tree** means **recursively traversing** the tree and **executing nodes** when their arguments are known. We have learned that there are problems with **non-functional** instructions and that we need **macros**, which have unevaluated arguments as their input.

What did we learn? (4)

- XO with tree-like programs requires **swapping** of sub-trees.
- Mutation requires **replacing a randomly chosen subtree** with a newly generated random program.
- Programs can also be composed of **multiple trees**, each representing a different component of the program. This allows the GP to **reduce** the **complexity** of solutions and to **discover** and **automatically** use “good” reusable subroutines. Of course, such structural changes require adaptation of genetic operators and initialisation routines.

[CE310] Evolutionary Computation and Genetic Programming

Riccardo Poli

Next time:
Bloat in GP

University of Essex
School of Computer Science and Electronic Engineering